Minimum XOR Path in a Grid

Daily Question - October 28, 2026

Problem Statement

You are given a 2D integer array grid of size m x n. Start at the top-left cell (0, 0) and reach the bottom-right cell (m - 1, n - 1). At each step, you may move only right or down.

The cost of a path is the bitwise XOR of all cell values along that path, including the start and end cells. Return the minimum possible XOR value among all valid paths.

Examples

1. Input: grid = [[1,2],[3,4]]. Output: 6. The two path XORs are 1 XOR 2 XOR 4 = 7 and 1 XOR 3 XOR 4 = 6.

2. Input: grid = [[6,7],[5,8]]. Output: 9. The two path XORs are 6 XOR 7 XOR 8 = 9 and 6 XOR 5 XOR 8 = 11.

3. Input: grid = [[2,7,5]]. Output: 0. The only path has XOR 2 XOR 7 XOR 5 = 0.

Constraints

1 <= m == grid.length <= 1000

1 <= n == grid[i].length <= 1000

m * n <= 1000

0 <= grid[i][j] <= 1023

Approach: Dynamic Programming over XOR States

All cell values fit in 10 bits. XOR cannot introduce a higher bit, so every reachable path XOR lies between 0 and 1023. Track all 1024 possible XOR values at each cell.

Do not keep only the minimum XOR at an intermediate cell: XOR does not preserve numerical order. For example, 1 < 2, but 1 XOR 2 = 3 while 2 XOR 2 = 0.

For a cell with value v, each reachable predecessor XOR x creates a reachable XOR x XOR v. The predecessors are the cell above and the cell to the left. Initialize the start cell with its own value. At the destination, scan the reachable XORs in increasing order and return the first one.

Rolling Array

Use dp[column][xor]. Before replacing dp[column], it holds states from the previous row (above). The already updated dp[column - 1] holds states from the current row (left). Build a fresh current array for each cell to avoid mixing old and new states.

Java Solution

```
class Solution {
    public int minCost(int[][] grid) {
        int rows = grid.length;
        int columns = grid[0].length;
        int xorStates = 1024;

        boolean[][] dp = new boolean[columns][xorStates];

        for (int row = 0; row < rows; row++) {
            for (int column = 0; column < columns; column++) {
                boolean[] current = new boolean[xorStates];
                int value = grid[row][column];

                if (row == 0 && column == 0) {
                    current[value] = true;
                } else {
                    for (int xor = 0; xor < xorStates; xor++) {
                        boolean fromAbove =
                                row > 0 && dp[column][xor];
                        boolean fromLeft =
                                column > 0 && dp[column - 1][xor];

                        if (fromAbove || fromLeft) {
                            current[xor ^ value] = true;
                        }
                    }
                }

                dp[column] = current;
            }
        }

        for (int xor = 0; xor < xorStates; xor++) {
            if (dp[columns - 1][xor]) {
                return xor;
            }
        }

        return -1;
    }
}
```

Correctness

The start state is correct because the path contains only the starting cell. For any later cell, every valid path enters from above or from the left. Extending every reachable predecessor XOR with the current value therefore produces all valid path XORs, and no invalid ones. By induction over row-major processing, the destination states are exactly the achievable path XORs. Returning the smallest reachable state gives the required minimum.

Walkthrough for Example 1

Reachable states: (0,0): {1}; (0,1): {3}; (1,0): {2}; (1,1): {7,6}. The minimum destination XOR is 6.

Complexity and Edge Cases

Time: O(m * n * 1024). Space: O(n * 1024) live auxiliary storage for the rolling DP and current array. The input grid is not modified.

For a single cell, return its value. For a single row or column, the DP follows the only possible path. If all cells contain zero, the answer is zero.